

Python프로그래밍 (112773) Lecture – 4



BAE HAN BYEOL
인공지능융합학과
Assistant Professor
Sep. 29, 2026

특수 메소드

- 파이썬에는 연산자(+, -, *, /)에 관련된 특수 메소드(special method)가 있다.
- 이들 메소드는 우리가 객체에 대하여 +, -, *, /와 같은 연산을 적용하면 자동으로 호출된다.

```
class Circle:
    ...
    def __eq__(self, other):
        return self.radius == other.radius

c1 = Circle(10)
c2 = Circle(10)
if c1 == c2:
    print("원의 반지름은 동일합니다. ")
```

특수 메소드

연산자	메소드	설명
<code>x + y</code>	<code>__add__(self, y)</code>	덧셈
<code>x - y</code>	<code>__sub__(self, y)</code>	뺄셈
<code>x * y</code>	<code>__mul__(self, y)</code>	곱셈
<code>x / y</code>	<code>__truediv__(self, y)</code>	실수나눗셈
<code>x // y</code>	<code>__floordiv__(self, y)</code>	정수나눗셈
<code>x % y</code>	<code>__mod__(self, y)</code>	나머지
<code>divmod(x, y)</code>	<code>__divmod__(self, y)</code>	실수나눗셈과 나머지
<code>x ** y</code>	<code>__pow__(self, y)</code>	지수
<code>x << y</code>	<code>__lshift__(self, y)</code>	왼쪽 비트 이동
<code>x >> y</code>	<code>__rshift__(self, y)</code>	오른쪽 비트 이동
<code>x <= y</code>	<code>__le__(self, y)</code>	less than or equal(작거나 같다)
<code>x < y</code>	<code>__lt__(self, y)</code>	less than(작다)
<code>x >= y</code>	<code>__ge__(self, y)</code>	greater than or equal(크거나 같다)
<code>x > y</code>	<code>__gt__(self, y)</code>	greater than(크다)
<code>x == y</code>	<code>__eq__(self, y)</code>	같다
<code>x != y</code>	<code>__neq__(self, y)</code>	같지않다

Lab:은행 계좌

- 2차원 공간에서 벡터(vector)는 (a, b) 와 같이 2개의 실수로 표현될 수 있다. 벡터 간에는 덧셈이나 뺄셈이 정의된다.

$$(a, b) + (c, d) = (a+c, b+d)$$

$$(a, b) - (c, d) = (a-c, b-d)$$

- 특수 메소드를 이용하여서 $+$ 연산과 $-$ 연산, `str()` 메소드를 구현해보자.

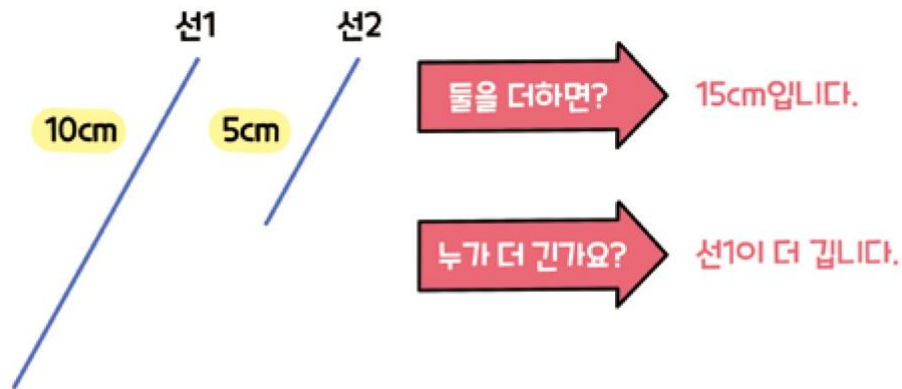
$$(0, 1) + (1, 0) = (1, 1)$$

```
class Vector2D :  
    def __init__(self, x, y):  
        self.x = x  
        self.y = y  
  
    def __add__(self, other):  
        return Vector2D(self.x + other.x, self.y + other.y)  
  
    def __sub__(self, other):  
        return Vector2D(self.x - other.x, self.y - other.y)  
  
    def __eq__(self, other):  
        return self.x == other.x and self.y == other.y  
  
    def __str__(self):  
        return '(%g, %g)' % (self.x, self.y)
```

```
u = Vector2D(0,1)  
v = Vector2D(1,0)  
w = Vector2D(1,1)  
a = u + v  
print(a)
```

2개의 선을 더하고 비교하는 프로그램을 만들어 봅시다.

그런데 파이썬에는 선(Line)이라는 데이터 형식은 없기 때문에 선이라는 객체를 만들어서 사용하려고 합니다. 우리가 만들 선의 속성으로는 길이를 지정하겠습니다. 메소드로는 서로 다른 2개 선을 더하면 길이가 합해지는 기능과 2개 선의 크기를 비교하는 기능을 만듭니다. 비교해서 어느 것이 더 긴지 알려주고, 짧은 선은 삭제하는 기능을 합니다.



실행 결과

10 길이의 선이 생성되었습니다.
5 길이의 선이 생성되었습니다.
두 선의 합 : 15
선1이 더 길니다.
5 길이의 선이 제거되었습니다.

1. lab10-02.py 파일을 만들고, 선(Line) 클래스를 생성하기. 우선 선 클래스에서 생성자를 만들기

```
## 클래스 선언
class Line :

    def __init__(self, length) :
        self.length = length
        print(self.length, '길이의 선이 생성되었습니다.')
```

2. 선을 삭제할 del(객체) 코드를 만나면 실행되는 소멸자 __del__() 메소드를 생성하기

```
def __del__(self) :
    print(self.length, '길이의 선이 제거되었습니다.')
```

3. 두 선의 객체를 더할 때 실행되는 __add__() 메소드를 만들기

```
def __add__(self, other) :
    return self.length + other.length
```

4. `__lt__()`는 두 선의 객체를 작다(<) 연산자로 비교할 때 실행됨

- self.length가 짧으면 True를, other.length가 짧으면 False를 반환함

```
def __lt__(self, other) :  
    return self.length < other.length
```

5. `__eq__()` 메소드는 두 선의 길이를 비교함

- 실행됩니다. self.length가 짧으면 True를, other.length가 짧으면 False를 반환함

```
def __eq__(self, other) :  
    return self.length == other.length
```

6. 2개의 선 객체를 만들기

```
## 메인 코드 부분  
line1 = Line(10)  
line2 = Line(5)
```

7. 2개 선의 길이 합을 구하기 위해 객체의 합계를 출력하기

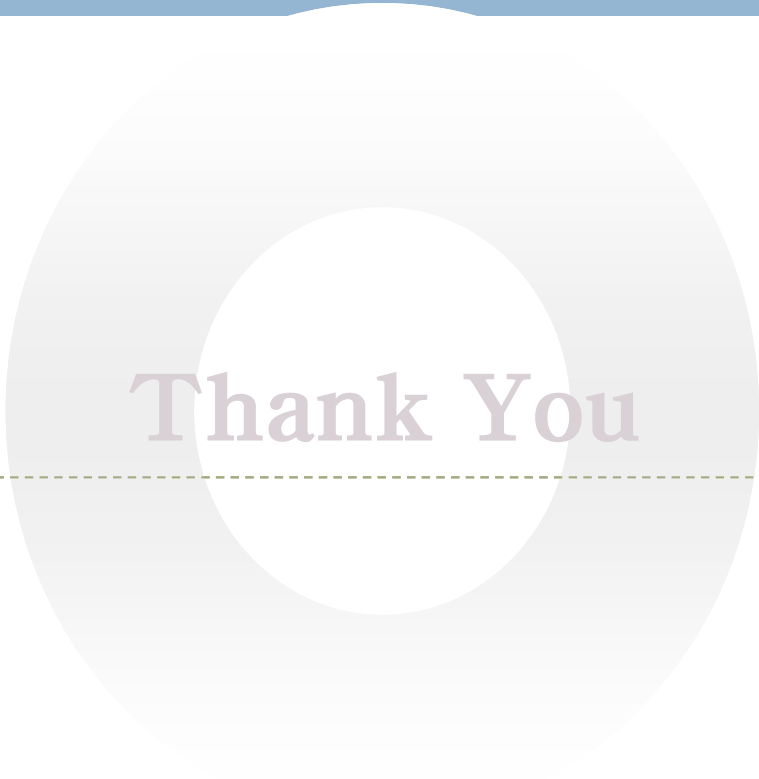
```
print('두 선의 합 : ', line1 + line2)
```

8. 두 선을 비교하기 위해 `__lt__()` 및 `__eq__()` 메소드를 호출함

- 이때 두 선을 비교하여 더 짧은 선은 삭제함

```
if line1 < line2 :  
    print('선2가 더 길니다.')  
    del(line1)  
elif line1 == line2 :  
    print('두 선의 길이가 같습니다.')  
else :  
    print('선1이 더 길니다.')  
    del(line2)
```

9. <Ctrl>+<S>를 눌러서 변경된 내용을 저장하고, <F5>를 눌러 실행 결과 확인하기



Thank You

